

1. Uvod

Razumevanje izvršavanja programa na nivou asemblerskog jezika podrazumeva praćenje veze između instrukcija, sadržaja registara, memorije i redosleda izvršavanja. Konačan rezultat programa ne pokazuje sva međustanja kroz koja je procesor prošao da bi do njega došao. Zbog toga je za analizu rada programa značajno posmatrati ne samo izlazne vrednosti, već i promene koje nastaju izvršavanjem pojedinačnih instrukcija.

Motivacija za ovaj rad jeste izrada okruženja u kome se kratak asemblerski program može napisati, izvršiti i analizirati uz neposredan uvid u stanje simuliranog procesora. Na primeru petlje koja sabira niz brojeva, korisnik treba da može da prati promenu zbira, vrednost brojača i trenutak u kome uslov grananja prestaje da bude ispunjen. Takav način rada namenjen je povezivanju tekstualnog zapisa programa sa njegovim izvršavanjem, bez potrebe da se za svaki korak ručno vodi evidencija svih promena.

Predmet rada vezan je za RISC-V, porodicu arhitektura skupa instrukcija, odnosno pravila koja određuju instrukcije i stanje procesora vidljivo programu. Specifikacija RISC-V razdvaja osnovni celobrojni skup instrukcija od dodatnih proširenja, pri čemu RV32I označava osnovnu varijantu sa 32-bitnim celobrojnim registrima [1]. U ovom radu razmatra se odabrani podskup instrukcija te varijante, dovoljan za primere aritmetičkih i logičkih operacija, pristupa memoriji i promene toka izvršavanja.

Predmet praktičnog dela rada jeste razvoj veb aplikacije koja objedinjuje editor asemblerskog koda, parser, assembler i simulator izvršavanja. Korisnik unosi program u tekstualni editor, nakon čega aplikacija proverava njegov zapis i prevodi ga u internu reprezentaciju pogodnu za izvršavanje. Ako je program ispravan, prelazi se u režim u kome se može upravljati izvršavanjem i pratiti stanje registara i memorije.

Osnovni cilj je realizacija preglednog alata koji omogućava izvršavanje jedne instrukcije, nastavak izvršavanja do tačke prekida i izvršavanje do završetka programa. Dodatni ciljevi obuhvataju prikaz promena stanja posle korisničke akcije, prijavljivanje grešaka sa vezom ka izvornom kodu i vraćanje simuliranog procesora u početno stanje. Pri tome je potrebno sačuvati vezu između instrukcije koju simulator izvršava i odgovarajuće linije koju korisnik vidi u editoru, kako bi praćenje programa ostalo razumljivo i nakon prevođenja pseudo-instrukcija.

Obim rešenja ograničen je na 15 instrukcija i pet pseudo-instrukcija. Podržane instrukcije obuhvataju sabiranje i oduzimanje, bitovske logičke operacije i pomeranja, učitavanje i upis memorijske reči, uslovno grananje i skokove. Model stanja sadrži 32 registra širine 32 bita, programski brojač i memoriju podataka veličine 4096 bajtova, dok se pseudo-instrukcije u fazi asembliranja prevode u podržane prave instrukcije.

Simulator predstavlja model izvršavanja na nivou instrukcija, a ne model unutrašnje hardverske organizacije procesora. Asemlirani program čuva se kao niz strukturiranih instrukcija, odvojeno od memorije podataka, bez generisanja binarnog mašinskog koda. Ne modeluju se protočna obrada, keš memorija niti trajanje instrukcija u taktovima, pa prikazani rezultati opisuju promene programskog stanja, a ne performanse stvarnog procesora.

Postupak izrade zasnovan je na razdvajanju obrade izvornog koda, izvršavanja i korisničkog interfejsa. Parser izdvaja strukturu linija i operanada, a assembler proverava da li zapis odgovara podržanim instrukcijama i u dva prolaza razrešava labele i priprema program. Model procesora potom izvršava pripremljene instrukcije i menja registre, memoriju i programski brojač, nezavisno od načina na koji se te promene prikazuju korisniku.

Za implementaciju je korišćen TypeScript, dok je korisnički interfejs realizovan veb tehnologijama HTML i CSS i neposrednim radom sa objektnim modelom dokumenta, odnosno DOM-om. Vite se koristi za razvojno okruženje i pripremu aplikacije za isporuku, a Vitest za automatizovane testove. Logika parsera, assemblera i modela procesora ne zavisi od elemenata veb stranice, što omogućava njenu proveru odvojeno od prikaza.

Interakcija sa programom organizovana je kroz režim pisanja i režim izvršavanja. U režimu izvršavanja komanda Step izvršava jednu instrukciju, dok Run nastavlja rad do završetka, tačke prekida ili greške. Prikaz izdvaja sledeću instrukciju, registre u koje je upisivano i memorijske reči koje su menjane, a Reset omogućava ponovno izvršavanje učitanoog programa od početnog stanja.

Prijavljivanje grešaka obuhvata i obradu izvornog teksta i samo izvršavanje. Neispravan zapis instrukcije ili nedefinisana labela sprečavaju pokretanje programa, dok nedozvoljen pristup memoriji ili neispravno odredište skoka prekidaju izvršavanje. Uvedeno je i ograničenje broja instrukcija u jednom pozivu komande Run, čime se ograničava trajanje izvršavanja programa koji ne dolazi do završetka ili tačke prekida.

Provera rešenja zasniva se na automatizovanim testovima komponenti i ručnoj proveru korisničkog interfejsa. Testovi obuhvataju ispravne programe i granične slučajeve, kao što su prekoračenje dozvoljenog opsega konstante, 32-bitno prelivanje, neporavnat pristup memoriji i ponašanje grananja i skokova. U radu se rezultati ovih provera razmatraju kao dokazi o ponašanju implementiranih funkcija u ispitanim slučajevima, bez tvrdnje da predstavljaju formalni dokaz ispravnosti ili procenu uticaja alata na uspeh u učenju.

Rezultat praktičnog dela jeste integrisana aplikacija u kojoj se program može pratiti od unosa izvornog teksta do promene stanja simuliranog procesora. Doprinos rada je u projektovanju, povezivanju i proveru komponenti koje ostvaruju zadate funkcionalnosti u okviru jasno ograničenog skupa instrukcija. Namenjena je demonstraciji i analizi kraćih asemblerskih programa, dok podrška celokupnom skupu instrukcija i složenijem razvojnom okruženju ostaje izvan obima ovog rešenja.

Rad je organizovan u osam poglavlja. Posle uvoda, drugo poglavlje daje pregled postojećih rešenja i kriterijuma za njihovo poređenje, dok treće objašnjava teorijske osnove potrebne za razumevanje simulatora. Četvrto poglavlje predstavlja korišćene tehnologije, a peto arhitekturu i implementacione detalje sistema.

U šestom poglavlju prikazane su funkcionalnosti aplikacije kroz primer korišćenja i praćenja izvršavanja programa. Sedmo poglavlje opisuje postupak testiranja, rezultate i ograničenja sprovedene provere. Osmo poglavlje donosi zaključak o ostvarenim ciljevima i razmatra moguća unapređenja rešenja.

Reference

- [1] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, izdanje 20260120, Introduction: RISC-V ISA Overview. <https://docs.riscv.org/reference/isa/v20260120/unpriv/intro.html> [pristupljeno 28.09.2026.].